Multi-Label Toxic Comment Classification with an Eight-Layer RoBERTa Architecture

Original author: Mohammad Albatayneh wrote the original code for this individual project, with AI-assisted corrections. New portfolio additions are AI-assisted.

Review date: 13 October 2026. **Evidence status:** historical notebook outputs, a later real-data source audit, a completed full-cohort CPU reference experiment, and verified engineering repairs. The new CPU scores use a grouped exploratory internal protocol. Historical neural performance has not been independently reproduced; full-data neural training remains unexecuted.

Abstract

Online comments may contain several forms of abusive language simultaneously, making toxicity detection a multi-label classification task. This project examines an eight-layer RoBERTa architecture for six toxicity categories using an upstream preprocessed version of the Jigsaw Toxic Comment dataset. The encoder was randomly initialized, while tokenization used the pretrained `roberta-base` tokenizer. The original notebooks randomly partitioned the labeled data into 80% training and 20% validation sets. At the final epoch, the retained five-epoch output reported macro ROC-AUC 0.9792 and macro F1 0.4391 at a 0.5 score threshold; minimum validation loss selected a different checkpoint at epoch three. Rare-label recall was high, but low precision limited F1. Later supplied CSVs support a new CPU TF-IDF/logistic-regression experiment on 159,564 eligible rows with grouped 70/15/15 partitions. Its 23,955-row exploratory internal test reports macro AP 0.6502, macro ROC-AUC 0.9811, and macro F1 0.6346 at frozen validation-selected thresholds. The changed protocol prevents a controlled performance comparison with the historical neural output. The repository revision repairs checkpoint consistency, input validation, duplicate-aware partitioning, threshold selection, and frozen evaluation. Full-data neural training has not been rerun, and no deployment, interpretability, or superiority claim is made.

1. Task and motivation

A comment can carry any combination of six binary labels: toxic, severe toxic, obscene, threat, insult, and identity hate. The implemented label order is `identity_hate`, `insult`, `obscene`, `severe_toxic`, `threat`, and `toxic`. Each model therefore produces six scores rather than a single mutually exclusive class. A clean comment has no positive labels.

The historical work focuses on class imbalance and a smaller transformer configuration. Evaluation must account for both missing harmful content and incorrectly flagging benign content. Precision, recall, per-label F1, and score-based ranking metrics describe these different behaviors. The intended use is research and reproducible experimentation; automated moderation suitability requires additional evidence.

2. Data provenance and the historical split

The supplied notebooks download `fizzbuzz/cleaned-toxic-comments` with KaggleHub and load `train_preprocessed.csv`. They rename `severe_tox` to `severe_toxic` and retain rows whose `set` field equals `train`. The saved download output in `model_FINAL_Report.ipynb`, cell 2, identifies dataset version 1, but the download call does not pin a version. This is a cleaned derivative of the original Jigsaw challenge data, rather than a direct load of the official training CSV.

The original report describes 159,571 labeled comments. The initial notebook/report uploads contained no source CSV. A mirror CSV supplied later contains 159,571 rows with complete binary labels, as recorded in the data audit. Its relationship to the exact historical download bytes remains unverified.

Seven comments are normalized-empty; the reviewed new-run policy drops them explicitly, retaining 159,564 rows. There are 93 conflicting normalized-text groups comprising 213 rows; their original annotations are retained together, without label merging. The notebooks themselves implement tokenization, truncation, and padding; they do not contain lowercasing, markup-removal, or whitespace-cleaning functions. Upstream cleaning remains unresolved.

The original split samples 80% of the labeled rows with `random_state=42` and assigns the complement to validation. It does not stratify the six labels or keep duplicate comments together. Validation is inspected after every epoch and used to select a checkpoint by loss. No original challenge test-set evaluation or independent final test result is recorded.

The later challenge-test mirror contains 153,164 rows, with every cell of its six target columns blank. It cannot supply a performance estimate. The two supplied CSVs share 636 nonempty normalized-text groups despite disjoint IDs, so separate files do not establish text independence. No challenge-test scoring is reported. The source audit contains aggregate counts and file identities, not comment examples or predictions.

3. Historical model and training procedure

The tokenizer is loaded from `roberta-base`. The encoder is created directly from a custom `RobertaConfig`; pretrained encoder weights are not loaded. The distinction matters because the experiment trains a randomly initialized transformer for the task rather than fine-tuning a pretrained language model.

Component	Historical configuration
Encoder blocks	8
Hidden dimension	512
Attention heads	8
Feed-forward intermediate dimension	2048
Hidden/attention dropout	0.1 / 0.1
Positional embedding table size	512
Actual tokenized input cap	256, including special tokens
Outputs	Six logits, followed by sigmoid for scoring

The standard Hugging Face sequence-classification head uses the first `<s>` token representation, dropout, a 512-dimensional dense layer, tanh, dropout, and a projection to six logits. Binary cross-entropy with logits is computed separately from scoring. Each label receives a positive-class weight equal to its training-partition negative-to-positive ratio. A label is predicted when its sigmoid score is strictly greater than 0.5. No threshold search is recorded in the original notebooks.

The five-epoch experiment uses AdamW with learning rate `4 × 10^-5`, weight decay `1 × 10^-2`, batch size 256, and a cosine schedule with 10% warmup. CUDA autocast and gradient scaling are enabled. TensorBoard records training loss, learning rate, validation loss, macro ROC-AUC, and macro F1. The code saves a checkpoint whenever validation loss improves and configures early stopping with patience three. This run completes all five scheduled epochs.

4. Historical results

The following tables reproduce saved notebook outputs from the historical metrics record. They are not newly computed measurements. Cell references use zero-based notebook indices. `model_FINAL_Report.ipynb` and `modelXXXX(1).ipynb` retain the same five-epoch metric sequence and do not provide independent replication evidence.

Notebook	Recorded checkpoint	Epoch	Validation loss	Macro ROC-AUC	Macro F1 at > 0.5
`model_FINAL_Report.ipynb`	Selected by validation loss	3	0.3482	0.9790	0.3900
`model_FINAL_Report.ipynb`	Last epoch	5	0.3824	0.9792	0.4391
`Untitled10.ipynb`	Selected by validation loss	4	0.3498	0.9801	0.3818
`Untitled10.ipynb`	Last epoch	7	0.4081	0.9810	0.4459
`modelXXXX(1).ipynb`	Selected by validation loss	3	0.3482	0.9790	0.3900
`modelXXXX(1).ipynb`	Last epoch	5	0.3824	0.9792	0.4391

These are historical validation outputs, not a new run or held-out test results. FINAL_Report and modelXXXX share the same saved five-epoch history; they are not independent replications.

Five-epoch run: retained validation output

Source: `model_FINAL_Report.ipynb`, cell 6; matching output in `modelXXXX(1).ipynb`, cell 6. F1 uses score > 0.5.

Epoch	Training loss	Validation loss	Macro ROC-AUC	Macro F1
1	0.7962	0.3941	0.9720	0.3688
2	0.3543	0.3679	0.9766	0.4357
3	0.2803	0.3482	0.9790	0.3900
4	0.2409	0.3735	0.9795	0.4306
5	0.2197	0.3824	0.9792	0.4391

Per-label metrics: retained final-epoch output

Source: `model_FINAL_Report.ipynb`, cell 9. The cell retains printed output with a null execution count and uses final-epoch prediction arrays from cell 6. No fresh evaluation was performed during this review.

Label	Precision	Recall	F1
Toxic	0.572	0.900	0.700
Obscene	0.539	0.959	0.690
Insult	0.440	0.945	0.601
Severe toxic	0.203	0.975	0.336
Identity hate	0.124	0.911	0.218
Threat	0.048	0.853	0.090
Macro average	0.321	0.924	0.439

Alternative run: retained validation output

Source: `Untitled10.ipynb`, cell 6. Learning rate `3 × 10^-5`, warmup 15%, ten epochs planned, seven recorded. F1 uses score > 0.5.

Epoch	Training loss	Validation loss	Macro ROC-AUC	Macro F1
1	1.0842	0.6579	0.9570	0.4369
2	0.4316	0.3910	0.9743	0.3886
3	0.3214	0.3514	0.9781	0.3716
4	0.2653	0.3498	0.9801	0.3818
5	0.2259	0.3610	0.9811	0.4221
6	0.1984	0.4412	0.9808	0.4689
7	0.1756	0.4081	0.9810	0.4459

Checkpoint identity

For the five-epoch run, epoch three has the minimum recorded validation loss, 0.3482, with macro F1 0.3900. Epoch five has the highest recorded F1, 0.4391, but validation loss 0.3824. Epoch four has the highest recorded ROC-AUC, 0.9795. These are different selection criteria and checkpoints.

Cell 7 exports the final in-memory model before loading the minimum-loss checkpoint for inference. Loading those weights does not replace the prediction arrays retained from epoch five. Consequently, the per-label report in cell 9 and the original ROC plot describe final-epoch predictions, while subsequent demonstration inference uses the epoch-three model. The original report incorrectly calls 0.382 the best loss and does not make this distinction.

In the alternative run, early stopping occurs after epoch seven. Its minimum-loss checkpoint is epoch four, while the highest recorded F1, 0.4689, occurs at epoch six. Because learning rate, warmup, and planned duration all differ, this comparison cannot isolate a cause of improvement. It also does not supply final-test evidence.

Interpretation

The final-epoch five-epoch run has high macro ROC-AUC, but macro precision is 0.321 and macro F1 is 0.439. Threat and identity hate have recalls of 0.853 and 0.911, respectively, alongside precisions of only 0.048 and 0.124. Their low F1 scores therefore reflect a substantial false-positive burden at the fixed threshold. The original conclusion that minority-label recall is modest is inconsistent with the recorded metrics.

ROC-AUC measures score ranking over thresholds; F1 measures binary decisions at a particular threshold. High ROC-AUC does not establish high precision, calibrated probabilities, or operational readiness. Macro F1 averages the six per-label F1 values and should not be recomputed as the harmonic mean of macro precision and macro recall.

The notebooks' ten hand-selected comments are qualitative demonstrations. Some benign examples receive several positive labels. These observations motivate error analysis, but the examples are neither an independently annotated benchmark nor a basis for a new accuracy estimate.

5. Revised repository methods

Status: integration and bounded engineering checks completed. This section describes the repaired implementation and evaluation protocol. The real CPU experiment is reported separately in Section 6; full-data neural training has not been executed. A tiny offline neural smoke run verifies mechanics with a synthetic tokenizer and reduced architecture, not the historical model's predictive performance.

Data validation and partitions

The revised loader supports the official training CSV and the historical mirror schema. It validates nonempty text and all six binary labels, handles the `severe_tox` alias, and filters mirror rows to `set == "train"`. It records source-file hashes, row counts, label supports, and duplicate/conflict summaries. Unknown labels are not silently converted to negatives.

Normalized exact-text groups are formed using Unicode NFKC normalization, case folding, and whitespace collapse. These operations serve grouping; model inputs retain their original text. Every group is assigned to one partition. The seeded target allocation is 70% training, 15% validation, and 15% internal test by **group count**, so realized row percentages may differ. Conflicting labels within an exact-text group require an explicit handling decision. This method is not multilabel stratification and does not detect paraphrases or near duplicates.

This new split differs from the historical 80/20 row split. Its metrics must be reported as a new experiment, with the actual partition counts; they cannot be presented as a controlled improvement over the old scores. The source cohort was already explored historically, so its internal test is exploratory rather than a pristine independent evaluation. Exact-text grouping does not establish author, discussion, temporal, or near-duplicate separation.

Reference model and repaired neural model

The reference pipeline uses word unigram/bigram TF-IDF fitted only on training text and six one-vs-rest, class-balanced logistic regressions. Its fixed settings include up to 50,000 TF-IDF features, the liblinear solver, `C=4`, and seed 42. The completed run provides a reproducible reference for a future same-split neural comparison.

The repaired neural path retains the original randomly initialized eight-layer, 512-dimensional RoBERTa architecture and pretrained tokenizer. It uses smaller microbatches with gradient accumulation to target an effective batch size of 256, workers zero by default, CUDA-only mixed precision, finite-value checks, and gradient clipping. Loss weights are computed from training labels only. Checkpoint selection remains based on validation loss, but the selected weights are reloaded before export and fresh validation predictions. Reload checks are intended to verify that the saved model reproduces the in-memory prediction path.

Thresholds and final evaluation

Label thresholds are selected using validation F1 on a fixed grid from 0.05 through 0.95. Ties choose the threshold nearest 0.5, then the higher threshold. A validation label with only one class uses 0.5. Neither TF-IDF fitting, class-weight calculation, checkpoint selection, nor threshold selection uses internal test labels.

Before a separate final-evaluation action, the dataset, split, configuration, model, and threshold identities are frozen. The evaluator is designed to reject changed artifacts and repeated evaluation within the same run. This guard supports a reviewable workflow; it cannot prevent a person from creating a separate run or adaptively changing research decisions after seeing test results.

The evaluation report includes per-label precision, recall, F1, positive support, ROC-AUC, and average precision (AP). AP follows the non-interpolated `average_precision_score` definition and is not mislabeled as trapezoidal PR area. Where a partition lacks a positive or negative class for a label, AP/ROC values and the full six-label macro AP/ROC are left undefined. Any mean over eligible labels is separately named and accompanied by coverage counts, preventing it from being mistaken for a full six-label average.

6. Completed real-data CPU reference experiment

The source audit and reviewed policies are fixed before fitting: exclude seven normalized-empty comments, retain conflicting duplicate annotations together, and ignore the derived `toxicity` target summary and IDs as features. No annotation labels are merged. The 159,564 eligible rows form these group-disjoint partitions:

Partition	Rows	Normalized-text groups
Training	111731	110772
Validation	23878	23736
Exploratory internal test	23955	23738

The recorded normalized-group overlap across partitions is zero. Each label has positive and negative support in validation and internal test, giving ranking coverage 6/6. TF-IDF, classifiers, and class weights use only training rows. The fixed threshold grid is applied only on validation; model and thresholds are frozen before the one internal-test evaluation. Decisions use `score >= threshold`, differing from the historical strict comparison.

Aggregate metrics under the new protocol

Sources: validation metrics and internal-test metrics. AP and ROC-AUC use continuous scores, so changing thresholds changes F1 without changing these ranking metrics. Validation metrics are selection diagnostics.

Partition	Threshold policy	Rows	Macro AP	Macro ROC-AUC	Macro F1	Micro F1
Validation	Fixed 0.5	23878	0.6549	0.9823	0.6087	0.7257
Validation	Validation-selected	23878	0.6549	0.9823	0.6414	0.7546
Internal test	Fixed 0.5	23955	0.6502	0.9811	0.6144	0.7330
Internal test	Validation-selected	23955	0.6502	0.9811	0.6346	0.7483

Per-label exploratory internal-test metrics

These thresholds were chosen on validation and frozen. Supports refer to the internal-test partition. Metrics are rounded to four decimals; exact values remain in the JSON record.

Label	Positive support	Threshold	AP	Precision	Recall	F1
identity_hate	224	0.70	0.4887	0.4206	0.6027	0.4954
insult	1196	0.60	0.7868	0.6655	0.8002	0.7267
obscene	1290	0.70	0.8928	0.8241	0.8209	0.8225
severe_toxic	261	0.80	0.4611	0.4329	0.6054	0.5048
threat	67	0.85	0.3884	0.4400	0.4925	0.4648
toxic	2299	0.75	0.8833	0.8383	0.7529	0.7933

Rare-label limitations remain visible: the internal test contains only 67 threat-positive rows and its AP is 0.3884. No uncertainty interval is supplied, so the result does not establish seed stability or a statistically significant model difference. The new model, split, and threshold protocol differ from the historical neural experiment. Its higher recorded F1 is not evidence of a controlled improvement over the old neural model.

Verification and reproducibility limits

The aggregate run verification records one internal-test evaluation, successful independent metric recomputation from saved private predictions, and zero normalized-group overlap. Private configuration, source snapshot, code hashes, environment metadata, fitted model, splits, and predictions remain outside publication. The public data summary preserves partition supports and exclusion policies.

The observed CPU development wall time was 47.10 seconds and its peak RSS was about 6.64 GiB with the three recorded numerical thread settings fixed to one. Evaluation took 7.13 seconds. These values describe this local experiment; they are not per-comment latency measurements, hardware-normalized benchmarks, or production-throughput claims.

The executed results-review notebook reads committed aggregates and renders historical and new-run evidence in separate sections. Local Jupyter sockets were unavailable in this sandbox, so every cell was executed top-to-bottom in a fresh IPython process with native rich-output capture. The notebook records that mode; the ordinary Jupyter-kernel protocol itself was not verified here.

7. Limitations and next experiments

The initial notebook/report uploads contained no source CSV or trained weights, and historical dependencies were not pinned. Later CSVs establish current source counts but do not identify the absent historical checkpoints or exact original input bytes. The audit can establish what the saved artifacts say, but it cannot independently reproduce the historical scores. One recorded split and a small number of experiments do not establish sensitivity to seed, generalization to new domains, or uncertainty bounds.

The supplied experiments contain no measured parameter comparison with BERT, timing benchmark, or LIME explanation. A smaller configured model and mixed precision do not themselves demonstrate faster inference or real-time suitability. The original literature entries were incomplete and unverified, so their numerical results are not reused as evidence of state-of-the-art performance.

The next substantive model comparison should train the repaired neural model on the same frozen source and grouped partitions as the CPU reference, with its selected saved model evaluated under the same metric definitions. The historical cohort has already been inspected, so a stronger generalization assessment additionally requires an independently reserved or newly collected labeled cohort. The supplied challenge-test mirror cannot supply that evaluation because its labels are blank and it shares normalized texts with training. Repeated seeds, independently reviewed errors, and an external domain assessment would help bound uncertainty. Explanations, augmentation, alternative losses, and latency measurement can be added as separately evaluated extensions. Synthetic fixtures validate software mechanics only.

8. Conclusion

The historical artifacts demonstrate a six-label training pipeline and strong validation score ranking, with moderate macro F1 and low rare-label precision at threshold 0.5. Correct reporting separates the final epoch from the checkpoint selected by minimum validation loss. The repository revision adds validated inputs, duplicate-aware partitions, validation-only decisions, consistent model export, and frozen evaluation. A completed real-data CPU reference reports macro AP 0.6502 and macro F1 0.6346 on an exploratory internal test under this new protocol. Full-data neural reproduction and a same-protocol comparison remain outstanding; no controlled gain or deployment readiness is established.

Evidence and methodological resources

- Historical metrics and provenance: exact retained values, cell/output references, and evidence limitations.

- Later supplied data audit: aggregate source counts, duplicate findings, blank challenge-test targets, and supplied-file identities.

- New CPU validation, internal test, partition supports, and run verification: distinct new-protocol aggregate evidence.

- Report correction ledger: page-by-page corrections to the supplied report.

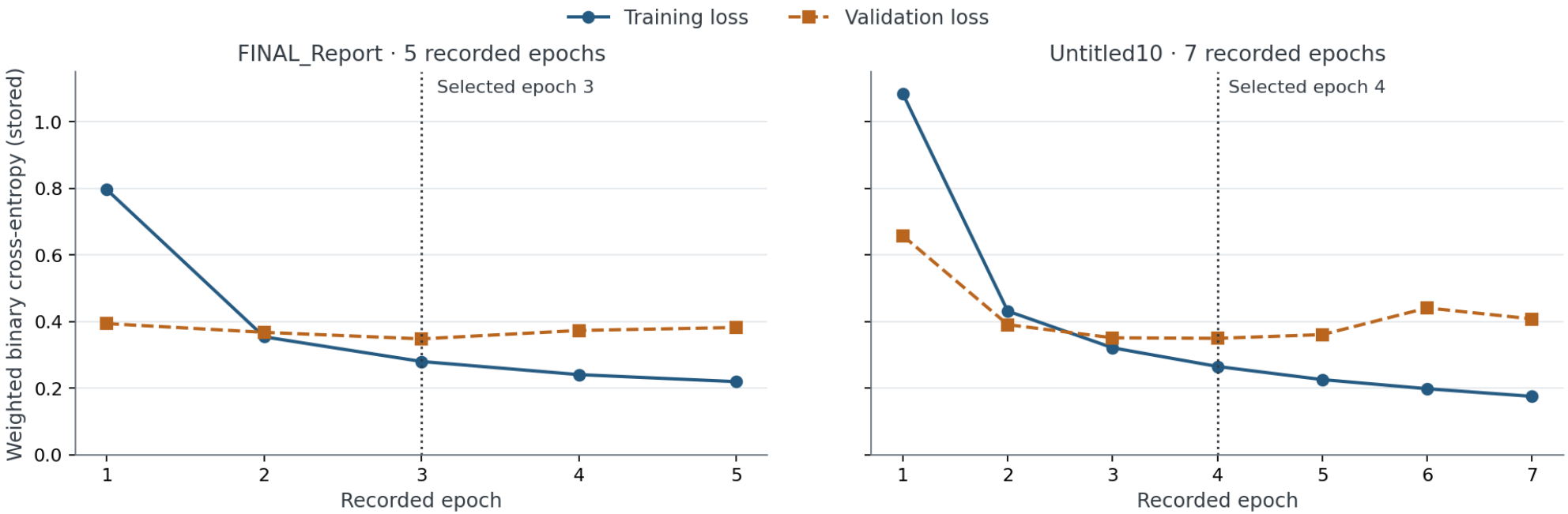
- Notebook audit and source manifest: implementation review and input identities.

- Original Jigsaw challenge data and FizzBuzz cleaned derivative: original/derivative provenance; redistribution rights require separate source review.

- Hugging Face RoBERTa documentation and sequence-classification source: model initialization and head structure.

- scikit-learn F1, ROC-AUC, average precision, and data-leakage guidance: metric definitions and training-only preprocessing.

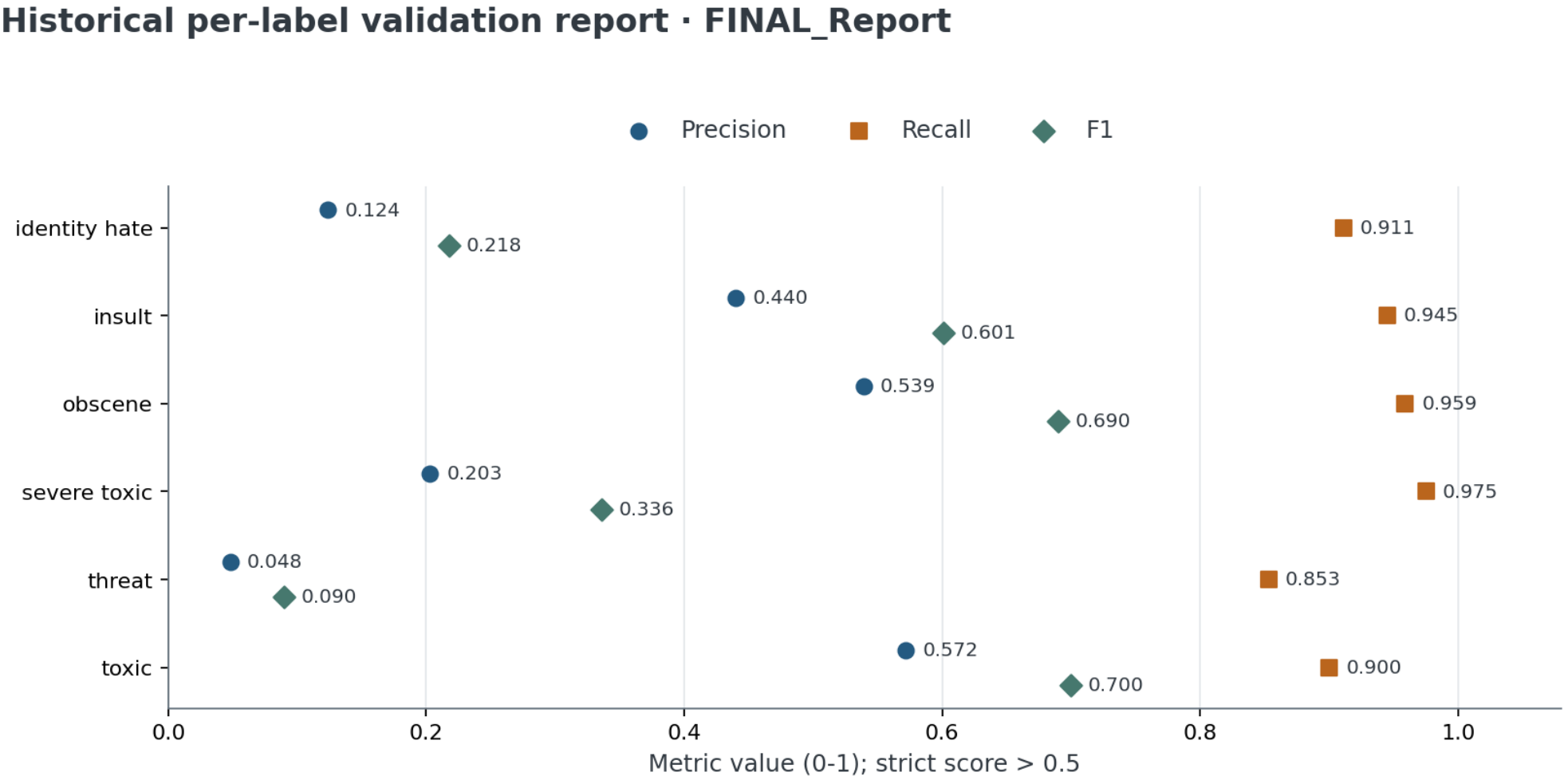
Figure 1. Historical loss trajectories



Source: original cell 6 outputs, rounded to four decimals. Validation repeatedly inspected; no rerun or held-out test.

Recorded notebook outputs, with the selected minimum-loss epochs marked. These are validation histories, not rerun measurements.

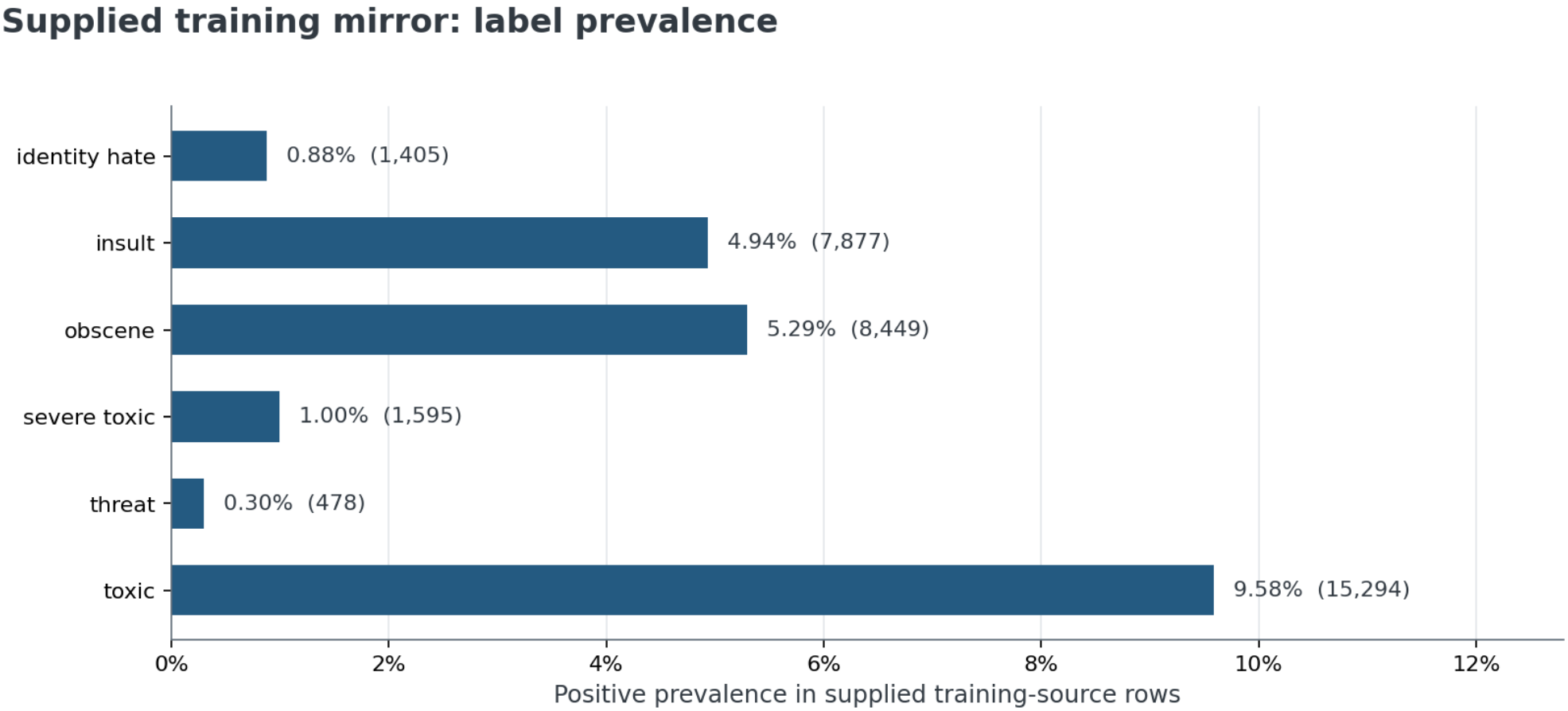
Figure 2. Historical precision, recall and F1



Source: stored cell 9 output (execution count null), rounded to three decimals; code uses last-epoch arrays.
No label supports, raw predictions, uncertainty intervals, or calibration evidence were supplied.

FINAL_Report's retained final-epoch per-label output. The selected checkpoint and these prediction arrays were not synchronized in the original workflow.

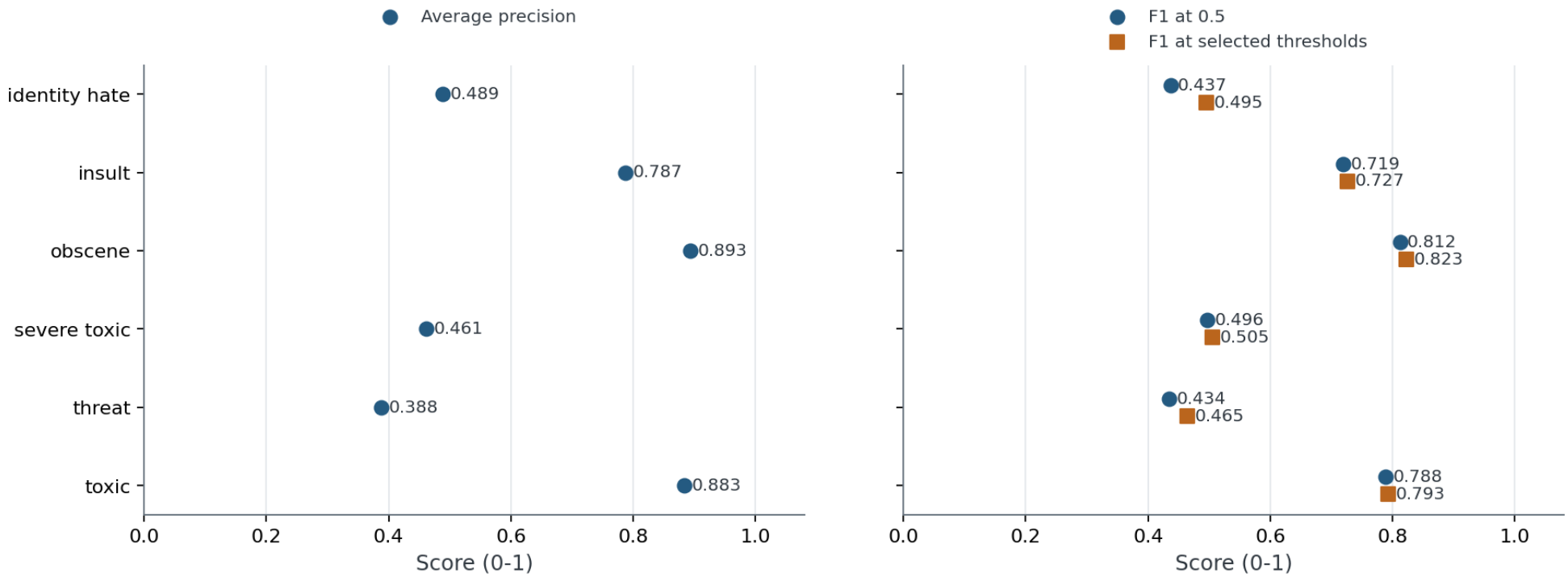
Figure 3. Later supplied source cohort



Source: reports/data_audit.json; denominator 159,571 source rows, before empty-text exclusions.
Labels overlap; these percentages do not sum to 100%. No subgroup or temporal independence is established.

Positive counts refer to the supplied training-source rows before the seven normalized-empty comments are excluded. The six labels overlap.

Figure 4. New CPU exploratory internal test



Population: 23,955 rows in a grouped exploratory internal test; original cohort was previously examined.
Thresholds selected on validation only. AP is not trapezoidal PR area. No confidence intervals are available.

A distinct 70/15/15 grouped protocol on a historically examined cohort. Thresholds are selected on validation; this figure does not compare the CPU model with the historical neural model.